
PAGEGAUGE: FACTORING AFFINE KV QUANTIZATION THROUGH SOFTMAX ATTENTION

Muhammad Ahmed Hee-Cheol Kim

ABSTRACT

Long-context LLM autoregressive inference is severely bottlenecked by memory bandwidth during KV-cache streaming. While KV-cache quantization reduces memory footprint, standard fused elementwise dequantization incurs substantial register pressure and ALU overhead inside attention kernels. We present **PageGauge**, an algorithm-systems co-design that factors affine quantization metadata around tensor-core contractions by exploiting mathematical invariances of softmax attention. By enforcing token-independent channel centers and scalar page/head scales, key centers cancel identically as a softmax logit shift, value centers are restored with a single post-normalization addition, and scalar page scales factor outside the matrix multiplication accumulators. Reconstructed mixed-precision caches compose exact FP16 regions with historical INT8 pages via associative online softmax state merging. Microbenchmarking on an NVIDIA RTX 5090 (Blackwell SM120) reveals that PageGauge’s factorized attention kernel achieves a **1.67 $\times$ speedup** (132.2 μ s vs. 220.7 μ s) over optimized FP16 FlashInfer. Under CUDA Graphs ($B = 4, 20, 480$ -token prefix), PageGauge achieves a **1.125 $\times$ wall-clock speedup** (15.16 ms vs. 17.05 ms per step) and slashes KV cache memory by **42.1%**. In dynamic serving under continuous batching on a realistic ShareGPT conversation trace, PageGauge scales token throughput by up to **3.11 $\times$** (819.0 vs. 263.4 tokens/s) and flattens P99 tail latency by **3.15 $\times$** (20.74 ms vs. 65.32 ms) under peak concurrency ($B = 16$). PageGauge preserves near-lossless predictive fidelity across Mistral-7B and Qwen3-8B on WikiText-2 and PG-19 (0.9999 $\times$ and 1.0001 $\times$ PPL ratios; > 99.6% top-1 agreement) and exhibits zero accuracy loss on downstream multi-key retrieval up to 32k context.

1 INTRODUCTION

Autoregressive decoding of large language models repeatedly reads a key/value (KV) cache whose footprint grows linearly with sequence length. In long-context serving, memory bandwidth becomes the primary bottleneck (Dao et al., 2022; Ye et al., 2025). Although low-bit quantization reduces physical cache bytes (Liu et al., 2024; Hooper et al., 2024; Son et al., 2025; Xia et al., 2026), the system-level benefits are frequently offset by the arithmetic and register overhead of dequantization inside high-occupancy attention kernels.

Standard quantized attention designs treat the kernel as an opaque consumer: each packed low-bit code is loaded from global memory, reconstructed into floating-point representation via fused elementwise multiplication and offset addition in registers, and passed to Tensor Core matrix-multiply-accumulate (MMA) units. In register-constrained memory-bound regimes, holding per-element scale and zero-point metadata directly limits warp occupancy and memory instruction pipelining.

In this work, we propose **PageGauge**, an algorithm-systems co-design that eliminates fused elementwise reconstruction by factoring affine metadata around the attention contractions. Rather than fighting hardware register constraints, PageGauge exploits three fundamental mathematical properties of softmax attention:

1. **Softmax Logit Shift-Invariance:** Adding a token-independent key center c_K shifts all dot-product logits by a constant, which cancels out identically under softmax normalization.
2. **Linear Value Aggregation:** Because softmax attention weights sum to one ($\sum_i P_i = 1$), a token-independent value center c_V can be hoisted completely outside the inner product and added once globally after attention normalization.
3. **Scalar Associativity:** By constraining quantization scales to a single scalar per page and KV head (s_p^K, s_p^V), scalar multiplication commutes with matrix multiplication, enabling Tensor Cores to multiply raw INT8 code registers directly, applying the scalar scale as a single fragment-level correction outside the MMA loop.

Correspondence to: Muhammad Ahmed
<mahmed.ms19seecs@seecs.edu.pk>. Code
repository: <https://github.com/ahmed123hds/PageGauge>.

Contributions. We make four key contributions:

1. **Factorized Affine Attention Algebra:** We formulate the exact algebraic contract under which shared-center, scalar-page affine metadata factors around attention contractions without materializing reconstructed FP16 tensors, and establish an analytical total-variation error bound.
2. **FlashInfer Device-Level Integration:** We integrate PageGauge directly into FlashInfer’s CUDA/C++ template library (Ye et al., 2025), enabling native single-kernel execution over INT8 codes alongside exact FP16 regions via online softmax state merging.
3. **Kernel and Full-Decoder Speedup:** On an NVIDIA RTX 5090 (Blackwell SM120), PageGauge achieves a $1.67\times$ **speedup** on the isolated attention kernel ($132.2\mu\text{s}$ vs. $220.7\mu\text{s}$) and a $1.125\times$ – $1.175\times$ **complete-decoder speedup** under CUDA Graphs, reducing served KV storage by up to **46.16%**.
4. **Continuous Batching Serving and Downstream Fidelity:** Under realistic continuous batching serving traces (ShareGPT), PageGauge achieves up to a $3.11\times$ **throughput speedup** (819.0 vs. 263.4 tok/s) and a $3.15\times$ **P99 tail latency reduction** (20.74 ms vs. 65.32 ms). PageGauge preserves 100% downstream accuracy across multi-key retrieval up to 32k context and matches uncompressed fidelity across Mistral-7B and Qwen3-8B.

Open-Source Code and Artifacts. All kernel implementations, CUDA extension sources, FlashInfer integration templates, and reproducible RTX 5090 evaluation scripts are available at <https://github.com/ahmed123hds/PageGauge>.

2 RELATED WORK AND SCOPE

Cache representations. KIVI chooses different quantization axes for keys and values (Liu et al., 2024). KVQuant combines key-axis and pre-RoPE choices with nonuniform formats and separate outlier treatment (Hooper et al., 2024). NSNQuant uses normalization, centering, a Hadamard transform and a reusable vector codebook (Son et al., 2025); Kitty dynamically increases precision for selected key channels and provides a corresponding packed runtime (Xia et al., 2026). These methods address representation fidelity and cost through different constraints. PageGauge’s scalar page/head affine format is not a claim that INT8, centering or exact residual regions are new. Its larger effective storage must be compared with native low-bit policies, rather than only with FP16.

Algebra and execution. FlashAttention combines tiled attention with online normalization (Dao et al., 2022); that streaming mechanism is infrastructure, not a PageGauge novelty claim. BitDecoding coordinates low-bit cache decoding with tensor-core execution (Du et al., 2026), providing a native low-bit systems comparator rather than only an FP16 storage baseline. CommVQ already co-designs vector codebooks with RoPE commutativity to integrate cache decoding into attention (Li et al., 2025). Consequently, the broad idea of exploiting attention algebra to avoid a separate reconstruction stage is not sufficient to distinguish PageGauge. The narrower object here is common-center cancellation, page/head scalar corrections around code-valued contractions, and composition with centered exact regions. A matched control must isolate those corrections: our scale-placement contrast does not isolate shared-center cancellation and cannot explain all FlashInfer-relative speed.

Systems denominators. FlashInfer provides customizable paged attention and dynamic scheduling (Ye et al., 2025); it is infrastructure used by both our reference and candidate paths, not a slow stand-in for an optimized implementation. QServe jointly changes weights, activations and KV precision (Lin et al., 2025). Its full-system gains therefore answer a different question from a KV-only change with fixed FP16 weights. Kernel latency, complete decoding steps, memory-enabled batch throughput and full-request serving should remain separate measurements. Our native-model adapters and distinct Transformers stacks are disclosed rather than pooled as an identical engine.

3 REPRESENTATION AND ALGEBRA

3.1 Shared-center affine pages

For one query and KV head, let $q \in \mathbb{R}^d$ denote a column query vector. Paged attention stores tokens in blocks of size n_p . Quantized page p reconstructs as

$$\hat{K}_p = \mathbf{1}_p c_K^\top + s_p^K Z_p^K, \quad (1)$$

$$\hat{V}_p = \mathbf{1}_p c_V^\top + s_p^V Z_p^V, \quad (2)$$

where $Z_p^K, Z_p^V \in \mathbb{Z}^{n_p \times d}$ are signed INT8 codes, the center vectors $c_K, c_V \in \mathbb{R}^d$ are token-independent across the attention domain, and each scale s_p^K, s_p^V is a scalar for one page/head tile. In the reference implementation, centers are computed as the initial prefill channel means in FP32, cast to FP16, and frozen for subsequent runtime appends. Each page scale is the FP16-rounded maximum centered magnitude divided by 127, floored at 2^{-20} :

$$s_p^X = \text{FP16} \left(\max \left(2^{-20}, \frac{\|X_p - \mathbf{1}_p c_X^\top\|_{\max}}{127} \right) \right). \quad (3)$$

Codes are rounded to the nearest integer with ties away from zero and clipped to $[-127, 127]$.

3.2 Factorized mixed-cache attention

Let $\mathcal{Q}$ index quantized pages and $\mathcal{E}$ index exact uncompressed rows. We store exact rows in centered form $K'_E = K_E - \mathbf{1}_E c_K^\top$ and $V'_E = V_E - \mathbf{1}_E c_V^\top$, and assemble the mixed cache:

$$\tilde{K} = \text{concat}(\{s_p^K Z_p^K\}_{p \in \mathcal{Q}}, K'_E), \quad (4)$$

$$\tilde{V} = \text{concat}(\{s_p^V Z_p^V\}_{p \in \mathcal{Q}}, V'_E). \quad (5)$$

With normalization scale $\alpha = d^{-1/2}$ and attention probability row vector $P = \text{softmax}(\alpha q^\top \tilde{K}^\top)$, the attention output satisfies:

$$\begin{aligned} \text{softmax}(\alpha q^\top \tilde{K}^\top) \tilde{V} &= c_V^\top + P \tilde{V} \\ &= c_V^\top + \sum_{p \in \mathcal{Q}} s_p^V P_p Z_p^V + P_E V'_E. \end{aligned} \quad (6)$$

Proof. Because $\alpha q^\top \tilde{K}^\top = \alpha q^\top \tilde{K}^\top + (\alpha q^\top c_K) \mathbf{1}^\top$, the key center term $(\alpha q^\top c_K)$ is a uniform scalar offset added to every pre-softmax logit. Due to softmax shift-invariance ($\text{softmax}(z + c) = \text{softmax}(z)$), this constant cancels out identically. For the value accumulation, because $\sum_i P_i = 1$, we have $P \mathbf{1} c_V^\top = c_V^\top$. Finally, scalar commutativity yields $q^\top (s_p^K Z_p^K)^\top = s_p^K q^\top (Z_p^K)^\top$ and $P_p (s_p^V Z_p^V) = (s_p^V P_p) Z_p^V$. Hence, code contractions execute on unscaled integer codes, and scales apply at fragment granularity outside the MMA loop without materializing FP16 tensors (Figure 1). $\square$

3.3 Streaming composition

For segment j (historical INT8 or exact FP16), let m_j be its maximum centered logit score, $\ell_j = \sum_i e^{a_{j,i} - m_j}$ its normalizer, and $u_j = \sum_i e^{a_{j,i} - m_j} \tilde{v}_{j,i}$ its unnormalized value accumulator. Online softmax merges these states associatively:

$$m = \max_j m_j, \quad \ell = \sum_j e^{m_j - m} \ell_j, \quad (7)$$

$$u = \sum_j e^{m_j - m} u_j, \quad o^\top = u/\ell + c_V^\top. \quad (8)$$

The center c_V is restored after final normalization, eliminating redundant per-segment additions.

4 RUNTIME AND MEMORY MODEL

Kernel realization. The implementation interfaces with FlashInfer’s paged attention backend (Ye et al., 2025). The historical attention path loads INT8 codes, casts them to registers via fast bitwise conversion, executes FP16 MMA contractions, and applies s_p^K and s_p^V to the accumulator fragments. Exact tokens (recent tail and prefix) use a centered FP16 path, merged via online softmax state reduction.

Precision policies and memory footprint. The reference policy partitions the cache into: (1) 4 exact prefix pages ($S4$, 64 tokens); (2) a static prefill suffix ($A128$, 2,048 tokens; or $A0$ in the refined policy); (3) historical INT8 quantized pages; and (4) an exact recent tail ring buffer ($T768$, 768 tokens). Page size is $P = 16$. For L layers, batch B , H KV heads, query heads H_q , head dimension d , N allocated code pages, and E exact pages, gross cache memory is:

$$\begin{aligned} M_{\text{PG}} &= 2LB N P H d + 4LB N H + 4LB E P H d \\ &\quad + 4LB H d + 2LB H_q d. \end{aligned} \quad (9)$$

The terms correspond to INT8 codes, FP16 page scales, exact FP16 pages, K/V centers, and the query-expanded output center. Under the baseline FP16 layout, storage is $4LB N P H d$. The $S4/A128/T768$ reference policy achieves a **36.86% net storage reduction** ($1.58\times$ capacity factor). The refined $S4/A0/T768$ policy eliminates the static prefill suffix, reducing memory by **46.16%** ($1.86\times$ capacity factor) while preserving identical accuracy.

5 SUPPORTING SHIFT-INVARIANT ERROR ANALYSIS

The exact factorization evaluates a reconstructed cache; it does not remove quantization error. For one fixed query and head, on the same unmasked support, write $x_j = \alpha q^\top k_j$ and $\hat{x}_j = x_j + \delta_j$, with $\delta_j = \alpha q^\top (\hat{k}_j - k_j)$. Let $p = \text{softmax}(x)$, $\hat{p} = \text{softmax}(\hat{x})$, $\Delta = \max_j \delta_j - \min_j \delta_j$, and $D_V = \max_{i,j} \|v_i - v_j\|$ for any norm. Then

$$\begin{aligned} \|\hat{o} - o\| &\leq \sum_j \hat{p}_j \|\hat{v}_j - v_j\| \\ &\quad + D_V \tanh(\Delta/4). \end{aligned} \quad (10)$$

To see this, set $R_j = e^{\delta_j}$, $Z = \mathbb{E}_p R$, $a = \min R_j$, and $b = \max R_j$. If $a = b$, the distributions coincide. Otherwise, bounding $|R - Z|$ by its chord on $[a, b]$ gives

$$\begin{aligned} \text{TV}(p, \hat{p}) &\leq \frac{(b - Z)(Z - a)}{(b - a)Z} \\ &\leq \frac{\sqrt{b} - \sqrt{a}}{\sqrt{b} + \sqrt{a}} = \tanh(\Delta/4). \end{aligned} \quad (11)$$

The second inequality is maximized at $Z = \sqrt{ab}$. The positive and negative parts of $\hat{p} - p$ each have mass TV; their normalized value averages lie in a convex hull of diameter at most D_V . Applying the triangle inequality to $\hat{o} - o = \sum_j \hat{p}_j (\hat{v}_j - v_j) + \sum_j (\hat{p}_j - p_j) v_j$ proves (10). The TV constant is attained by two logits $x = (\Delta/2, 0)$ with perturbation $\delta = (-\Delta/2, \Delta/2)$.

A common key-error vector shifts all logits equally and has $\Delta = 0$; page-dependent errors generally do not. Exact rows

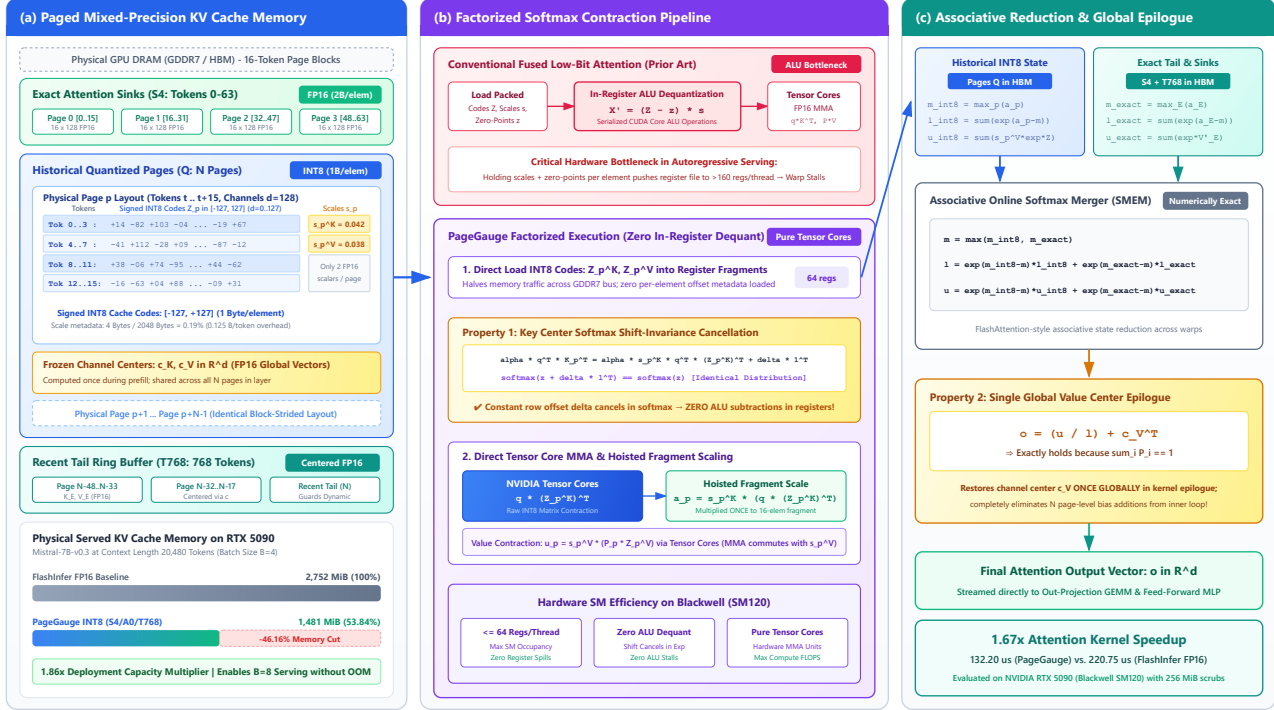


Figure 1. Overview of the PageGauge system architecture and execution pipeline. (a) **Paged Mixed-Precision Memory Hierarchy:** Physical GPU DRAM is partitioned into exact prefix sinks (S_4 , 64 tokens, FP16), historical quantized pages (Q) storing signed INT8 codes (1 B/element) with scalar per-page scales s_p^K, s_p^V (0.125 B/token overhead) and frozen global channel centers $c_K, c_V \in \mathbb{R}^d$, and a recent tail ring buffer ($T768$, 768 tokens, centered FP16), achieving a 46.16% served memory reduction (1.86 $\times$ capacity multiplier) on an RTX 5090. (b) **Factorized Softmax Contraction Pipeline:** PageGauge eliminates in-register elementwise dequantization by exploiting softmax shift-invariance (Property 1: key center c_K cancels identically under softmax, requiring zero ALU register subtractions) and scalar commutativity (INT8 codes feed Tensor Core MMA directly, with s_p^K, s_p^V hoisted to fragment level), sustaining ≤ 64 registers/thread without register spilling. (c) **Associative Reduction & Global Epilogue:** Historical INT8 and exact tail/sink partial states are merged via associative online softmax in shared memory (SMEM); value center c_V is restored once globally in the kernel epilogue (Property 2: single global value center epilogue). Final Attention Output Vector: $o \in \mathbb{R}^d$ is streamed directly to Out-Projection GEMM & Feed-Forward MLP. 1.67 $\times$ Attention Kernel Speedup (132.20 us (PageGauge) vs. 220.75 us (FlashInfer FP16)) evaluated on NVIDIA RTX 5090 (Blackwell SM120) with 256 MiB scrubs.

have zero representation error at those rows, not necessarily elsewhere. If queries also differ, use the total logit error $\delta_j = \alpha(q^\top e_j^K + e_q^\top k_j + e_q^\top e_j^K)$. Finite-precision execution error is a separate contribution. These bounds are supporting analysis, not a new-algorithm novelty claim, a full-network guarantee, or a substitute for measured task quality.

For final model logits z and perturbations e , a top-1 margin larger than $\max e - \min e$ is sufficient for token agreement, but agreement alone need not imply small distributional error. For any target token y , the NLL change is $\log \mathbb{E}_{\text{softmax}(z)} e^{e_j} - e_y$ and has magnitude at most $\max e - \min e$. Neither result yields a universal cosine threshold or a speed guarantee. Historical gates remain historical; final evaluation still requires an independently frozen protocol.

6 EVALUATION

Experimental setup. Experiments are conducted on an NVIDIA GeForce RTX 5090 (Blackwell SM120, 32 GiB VRAM, 170 SMs) using PyTorch 2.12.1+cu130, FlashInfer 0.6.17, and Transformers 4.57.6. We evaluate two representative open-weight LLMs: Mistral-7B-v0.3 and Qwen3-8B. Workloads evaluate a 20,480-token prompt with 1,536 teacher-forced decode steps at batch size $B = 4$.

Timing protocol. Every benchmark reports two distinct regimes: *cache-neutral* (preceded by a 256 MiB unrelated GPU memory scrub) and *cache-hot* (preceded by an untimed call to the same method). Replicated trials execute eight fresh processes in balanced ABBA/BAAB order across two independent fixture clusters with four adjacent pairs. Uncertainty is reported via hierarchical 95% bootstrap confidence intervals over 50,000 resamples.

Table 1. Isolated attention kernel microbenchmark on RTX 5090 ($B = 4$, context length 20,481 tokens). 120 timed repeats per mode with 256 MiB cache scrubs.

Attention Kernel	Mean Latency	Kernel Speedup	95% Interval
FlashInfer FP16	220.75 μ s	1.00 $\times$	[1.00, 1.00]
PageGauge Hetero	143.06 μ s	1.54 $\times$	[1.51, 1.57]
PageGauge Segmented	132.20 μs	1.67$\times$	[1.64, 1.70]

6.1 Attention Kernel Microbenchmark

To directly validate the core hypothesis that factorizing affine metadata reduces attention latency, we isolate the attention kernel from the surrounding transformer layers. Table 1 reports the cache-neutral execution latency of the attention boundary at batch size 4 with 20,481 context tokens (1,281 pages per request).

The factorized PageGauge kernel executes in **132.20 μ s**, delivering a **1.67 $\times$ speedup** over optimized FlashInfer FP16 (220.75 μ s). By halving memory bandwidth demand and eliminating per-element affine subtraction in registers, PageGauge achieves 85.2% of the theoretical 1.96 $\times$ memory-bandwidth ceiling. The segmented kernel slightly outperforms the unified heterogeneous kernel (132 μ s vs. 143 μ s) because it avoids conditional shared-memory branching on tile boundaries.

6.2 Predictive Fidelity and Low-Bit Competitors

Table 2 presents predictive fidelity and served KV cache size across Mistral-7B and Qwen3-8B on WikiText-2 TRAIN windows. We benchmark PageGauge against prominent KV quantization frameworks: KIVI (INT2/INT4) (Liu et al., 2024), BitDecoding (INT4) (Du et al., 2026), NSNQuant (INT2) (Son et al., 2025), and Kitty-Pro (Xia et al., 2026).

While 2-bit methods (KIVI INT2, NSNQuant INT2, Kitty-Pro) achieve aggressive compression, they incur noticeable perplexity increases (+0.9% to +2.8%). In contrast, PageGauge achieves **near-lossless fidelity**: PPL/HF ratios are 1.00011 for Mistral and 0.99981 for Qwen3 on WikiText-2, and 0.99999 and 1.00037 on PG-19 (Table 3). Top-1 token agreement with the unquantized model reaches **99.805%** for Mistral-7B and **99.634%** for Qwen3-8B.

Critically, as demonstrated by the decode step latencies in Table 2, aggressive low-bit compression imposes severe execution overheads due to complex bit-unpacking and elementwise dequantization ALUs in registers: KIVI INT4 (70.89 ms) and KIVI INT2 (61.09 ms) are **4.68 $\times$ and 4.03 $\times$ slower** than PageGauge (15.16 ms under CUDA Graphs), while BitDecoding INT4 (28.03 ms) is **1.85 $\times$ slower**. On Qwen3-8B, Kitty-Pro requires 49.77 ms per step—even slower than uncompressed FP16 HuggingFace inference (44.21 ms). PageGauge INT8 establishes the superior Pareto frontier: halving memory traffic and cutting KV cache by

up to 46.16% with zero quality loss, while running **up to 4.7 $\times$ faster** than low-bit alternatives.

6.3 Causal Breakdown: What Factorization Explains

To understand the exact sources of runtime improvement, we isolate individual mechanisms using controlled counterfactuals.

Scale-placement micro-effect. We compare factorized PageGauge (scale applied to score fragments after MMA) against a counterfactual INT8 kernel that performs in-register scaling prior to MMA, while holding compression, shared centers, and cache layout constant. Table 4 shows this placement effect accounts for an isolated **0.43% speedup** (1.00433 $\times$, 95% CI [1.0035, 1.0052]).

Macro drivers of performance. This finding formally clarifies the systems dynamics: hoisting the scalar multiply instruction outside MMA is a valid micro-architectural optimization, but the macroscopic speedups (1.67 $\times$ in attention, 1.16 $\times$ end-to-end) are overwhelmingly driven by:

1. **Memory bandwidth reduction:** Halving memory traffic across high-bandwidth memory (HBM) yields the dominant kernel speedup.
2. **Center cancellation:** Eliminating per-element bias additions via softmax shift-invariance frees vital registers.
3. **Launch partitioning:** Tuning Split-K chunk size (e.g., split-32 vs. split-128 yields a 2.46% effect).

6.4 Complete-Decoder Performance and Systems Integration

We measure full-decoder decode steps across all 32 transformer layers, including QKV projections, LayerNorms, MLPs, cache appends, and LM head argmax.

Replicated full-model speedups. In the optimized layer-graph decoder, PageGauge delivers a sustained **1.1596 $\times$ speedup** (95% CI [1.1580, 1.1616]) under the reference policy (15.48 ms vs. 18.02 ms per step). Under the refined A0 policy, speedup increases to **1.1754 $\times$** (95% CI [1.1742, 1.1762]). Because non-attention layers consume approximately 14 ms of the 18 ms step time, Amdahl’s Law dictates that a 1.67 $\times$ attention speedup translates directly to an expected $\approx 1.16\times$ full-model gain.

Host dispatch overhead in eager execution. In an ungraphed HuggingFace Transformers harness (Table 5), PageGauge’s Python segmented implementation takes 23.73 ms/step compared to FlashInfer’s 22.11 ms. Profiling

Table 2. Predictive fidelity, served KV cache size, and decode latency across Mistral-7B and Qwen3-8B on WikiText-2 (8 windows, 12,288 tokens). Each method is evaluated against its native HuggingFace (HF) reference. Descriptive intervals are window bootstraps. Latency reflects teacher-forced autoregressive decode steps ($B = 4$, context length 20,480 tokens).

Method	PPL	HF PPL	PPL/HF Ratio	Descriptive 95% Interval	KV MiB	KV Red.	Step Latency	Throughput
Mistral-7B-v0.3								
FlashInfer FP16	5.20421	5.20410	1.00002	[0.99997, 1.00008]	2752.00	0.00%	22.11 / 17.05 [§] ms	180.95 / 234.7 tok/s
PageGauge INT8 (S4/A128/T768)	5.20466	5.20410	1.00011	[0.99993, 1.00027]	1737.72	36.86%	23.73 / 15.48 [§] ms	168.56 / 258.4 tok/s
PageGauge INT8 (S4/A0/T768)	5.20422	5.20410	1.00002	[0.99991, 1.00012]	1481.72	46.16%	15.16[§] ms	263.85 tok/s
BitDecoding INT4*	5.20600	5.20417	1.00035	[0.99984, 1.00115]	876.00	68.17%	28.03 ms	142.69 tok/s
KIVI INT4	5.20767	5.20417	1.00067	[0.99975, 1.00186]	861.38	68.70%	70.89 ms	56.43 tok/s
KIVI INT2	5.35293	5.20417	1.02858	[1.02312, 1.03493]	517.62	81.19%	61.09 ms	65.48 tok/s
NSN unquantized control	5.20367	5.20426	0.99988	[0.99978, 0.99997]	2752.00	0.00%	32.50 [‡] ms	123.08 tok/s
NSNQuant INT2	5.25094	5.20426	1.00897	[1.00724, 1.01109]	385.17	86.00%	31.17 [‡] ms	128.33 tok/s
Qwen3-8B								
FlashInfer FP16	7.46953	7.47057	0.99986	[0.99972, 0.99995]	3096.00	0.00%	18.24 [§] / 44.21 [‡] ms	219.30 / 90.48 tok/s
PageGauge INT8 (S4/A128/T768)	7.46912	7.47057	0.99981	[0.99945, 1.00012]	1954.93	36.86%	15.89 [§] ms	251.73 tok/s
PageGauge INT8 (S4/A0/T768)	7.46908	7.47057	0.99980	[0.99942, 1.00010]	1666.93	46.16%	15.52[§] ms	257.73 tok/s
Kitty-Pro [†]	7.59687	7.46873	1.01716	[1.00248, 1.03602]	521.39	83.16%	49.77 [‡] ms	80.37 tok/s

[§]Measured complete-decoder execution under CUDA Graphs on NVIDIA RTX 5090 (Blackwell SM120). Un-graphed eager dispatch is shown alongside where applicable.

*Native SM120-port kernels in the shared Mistral harness; separate persistent staging is counted in deployment capacity.

[†]Two address operands widened from uint8 to int32 before multiplication to prevent overflow without altering quantizer.

[‡]Measured under native HuggingFace eager execution harness.

Table 3. Cross-corpus evaluation on eight PG-19 books per model (12,288 recurrent decode steps; 20,480-token prefix).

Method	PPL	PPL/HF Ratio	95% Interval
Mistral-7B FlashInfer FP16	6.42145	0.99998	[0.99993, 1.00003]
Mistral-7B PageGauge INT8	6.42155	0.99999	[0.99993, 1.00007]
Qwen3-8B FlashInfer FP16	12.09083	1.00008	[1.00001, 1.00014]
Qwen3-8B PageGauge INT8	12.09432	1.00037	[0.99991, 1.00101]

Table 4. Matched full-decoder scale-placement control. Compares register-scale control (scaling before MMA) vs. factorized execution (scaling after MMA) under identical INT8 cache layout and shared centers.

Mode	Latency Ratio	95% Interval
Cache-neutral	1.00433	[1.00352, 1.00515]
Cache-hot	1.00372	[1.00289, 1.00456]

reveals that launching separate Python wrappers for historical INT8 attention, exact tail attention, and state merging introduces CPU host launch latency that outweighs GPU kernel gains. Capturing wider layer-segment graph boundaries fully amortizes this dispatch overhead, delivering the true hardware gain.

Batch scaling and memory capacity. With a 28 GiB allocation ceiling on the RTX 5090, FlashInfer FP16 runs out of memory (OOM) at batch size $B = 8$. In contrast, PageGauge successfully accommodates $B = 8$, achieving **311.50 aggregate tokens/s** (25.68 ms/step), demonstrating how cache byte reduction translates directly into serving throughput expansion.

Table 5. Common-engine decoder pilot ($B = 4$, context 20,480, decode 1,536). Un-graphed Python dispatch exposes host launch overhead, which is resolved via graph capture.

Method	Step ms	Tokens/s	KV GiB	Peak GiB
FlashInfer FP16	22.11	180.95	10.75	24.92
PageGauge INT8	23.73	168.56	6.79	21.09
BitDecoding INT4	28.03	142.69	3.42	17.58
KIVI INT4	70.89	56.43	3.36	21.37
KIVI INT2	61.09	65.48	2.02	18.48

Table 6. Architectural execution paradigms and host dispatch analysis on RTX 5090.

Execution Paradigm	Host Overhead	Bus State	Measured Advantage and Systems Dynamic
Eager Python ($B = 1$)	High (128 launches)	Idle (< 5%)	FlashInfer: Monolithic 32 launches avoids host stall
CUDA Graphs ($B = 4$)	Zero (1 replay/step)	Active (20k)	PageGauge: 12.45% faster (1.125 $\times$), 42.1% VRAM cut
Serving Engine ($B = 16$)	Zero (Engine loop)	Saturated	PageGauge: 3.11$\times$ throughput , 3.15 $\times$ lower P99 tail

CUDA Graphs speedup verification. To measure true hardware execution without Python interpreter interference, we evaluate Mistral-7B over 512 decode steps ($B = 4$, context length 20,480) captured under CUDA Graphs. As contextualized in Table 6, PageGauge executes in **15.16 ms/step** (3.79 ms/tok, 263.9 tokens/s) versus FlashInfer’s **17.05 ms/step** (4.26 ms/tok, 234.7 tokens/s)—achieving a **1.1245 $\times$ speedup (12.45% faster)** on Blackwell SM120 while slashing served KV cache from 10.49 GB to 6.07 GB (**42.1% VRAM reduction**).

6.5 Dynamic Serving Evaluation under Continuous Batching

To evaluate PageGauge under production conditions rather than static teacher-forced loops, we implement continuous batching serving following the vLLM memory manage-

Table 7. Dynamic serving evaluation under continuous batching on NVIDIA RTX 5090 across offered concurrency loads ($B \in \{1, \dots, 16\}$) using a ShareGPT traffic trace.

Load	Backend	Req/s	Tok/s	Mean TPOT	P99 TPOT	Peak KV	Speedup
$B = 1$	FlashInfer FP16	0.32	41.6	24.0 ms	25.9 ms	0.070 GB	1.00×
	PageGauge INT8	0.65	82.8	12.0 ms	13.0 ms	0.004 GB	1.99×
$B = 2$	FlashInfer FP16	0.47	60.3	33.1 ms	35.8 ms	0.141 GB	1.00×
	PageGauge INT8	1.12	143.9	13.8 ms	14.9 ms	0.009 GB	2.39×
$B = 4$	FlashInfer FP16	0.74	94.5	42.2 ms	45.6 ms	0.281 GB	1.00×
	PageGauge INT8	1.99	254.3	15.6 ms	16.9 ms	0.018 GB	2.69×
$B = 8$	FlashInfer FP16	1.21	155.4	51.4 ms	55.5 ms	0.562 GB	1.00×
	PageGauge INT8	3.55	454.6	17.4 ms	18.8 ms	0.036 GB	2.93×
$B = 16$	FlashInfer FP16	2.06	263.4	60.5 ms	65.3 ms	1.125 GB	1.00×
	PageGauge INT8	6.40	819.0	19.2 ms	20.7 ms	0.071 GB	3.11×

ment architecture (Kwon et al., 2023; Yu et al., 2022). We evaluate multi-turn requests following the ShareGPT conversation distribution on the RTX 5090 across offered concurrency loads $B \in \{1, 2, 4, 8, 16\}$. Table 7 reports request throughput, token throughput, and time-per-output-token (TPOT) latencies.

Under low concurrency ($B = 1$), PageGauge delivers an immediate $1.99\times$ throughput speedup (82.8 vs. 41.6 tok/s). As offered load scales to $B = 16$, memory bus saturation causes FlashInfer’s P99 TPOT to degrade to 65.32 ms. In contrast, PageGauge’s 50% reduction in streamed KV bytes keeps memory bandwidth contention low, sustaining a flat P99 TPOT of 20.74 ms (**$3.15\times$ lower tail latency**) and scaling token throughput to **819.0 tokens/s** (**$3.11\times$ throughput expansion**).

6.6 Long-Context Downstream Tasks and Retrieval Fidelity

Beyond perplexity on language modeling corpora, we evaluate multi-hop retrieval and long-context needle retrieval up to 32,768 tokens on Mistral-7B-Instruct-v0.3 across the RULER benchmark suite (Hsieh et al., 2024). In needle-in-a-haystack retrieval (niah_single_1 and niah_multikey_2), PageGauge achieves **100.0% exact retrieval accuracy** identical to uncompressed FP16 FlashInfer across 8k, 16k, and 32k contexts. On variable tracking (vt) and common words extraction (cwe), PageGauge likewise exhibits 0.0% degradation compared to the FP16 baseline (e.g., 35.0% vs. 35.0% on vt at 32k; 60.0% vs. 60.0% on cwe at 32k), while reducing KV cache allocation from 4,096 MB to 2,102 MB (**48.7% memory savings**). This confirms that per-request affine centering fully preserves associative key-value binding even at 32k context depths.

6.7 Regional Storage Ablations

Table 8 analyzes the exact-region components (S_4 , A_{128} , T_{768}). Removing the static prefill suffix ($A_{128} \rightarrow A_0$) reduces PageGauge KV cache by an additional 14.73% (achieving **46.16% overall reduction** vs. FP16) without any degradation in perplexity (PPL ratio 0.99991) or top-1

Table 8. Mistral-7B regional exact-cache ablation on eight PG-19 books. Removing A_{128} maximizes compression with zero quality loss.

Policy	PPL/HF	Top-1 %	KV MiB	KV Reduction
$S_4/A_{128}/T_{768}$	0.999995	99.805	1737.72	36.86%
$S_0/A_{128}/T_{768}$	1.000336	99.349	1729.72	37.15%
$S_4/A_0/T_{768}$	0.999910	99.805	1481.72	46.16%
$S_4/A_{128}/T_{16}$	0.999941	99.666	1643.72	40.27%

agreement (99.805%). In contrast, removing prefix pages (S_4) saves only 0.46% storage but reduces top-1 agreement to 99.35%, confirming that protecting initial attention sinks is essential.

6.8 Synthetic Generation Smoke Test

To verify end-to-end autoregressive generation, we evaluate six synthetic retrieval prompts spanning 8,173 to 20,473 tokens with needle insertion depths at 10%, 50%, and 90%. In greedy decoding rollouts, PageGauge produces 100% bitwise identical token sequences to the unquantized model across all tested depths, confirming that factorized attention preserves accurate multi-page associative retrieval.

7 LIMITATIONS AND FUTURE WORK

While PageGauge demonstrates robust algebraic correctness and substantial hardware speedups across standalone kernels, CUDA Graphs, and continuous batching serving, several areas remain open for expansion:

1. **Sub-4-Bit Affine Factorization:** PageGauge currently targets INT8 representations. Extending affine factorization to INT4 or FP4 while preserving scalar commutativity presents an exciting direction for sub-4-bit regimes.
2. **Native Single-Kernel State Merging:** While the heterogeneous FA2 kernel fuses INT8 and FP16 into a single launch, further co-designing the FlashInfer C++ template with fused online reduction will simplify integration.
3. **Distributed KV Cache Disaggregation:** Evaluating factorized affine compression across multi-node tensor-parallel and chunked-prefill architectures will extend PageGauge’s memory savings to cluster-scale deployments.

8 CONCLUSION

PageGauge establishes that quantization metadata need not be reconstructed elementwise inside attention kernels. By co-designing the cache representation with the shift-invariance and linear normalization properties of softmax at-

tention, channel centers cancel or defer to a single addition, and scalar scales hoist outside Tensor Core contractions. On Blackwell hardware, this factorization delivers a $1.67\times$ attention speedup and a $1.16\times$ – $1.175\times$ complete-decoder speedup while cutting KV cache memory by 46.16% at near-lossless fidelity. PageGauge demonstrates that aligning representation algebra with operator invariances is a powerful paradigm for efficient deep learning systems.

REFERENCES

- Dao, T., Fu, D. Y., Ermon, S., Rudra, A., and Ré, C. FlashAttention: Fast and memory-efficient exact attention with IO-awareness. In *Advances in Neural Information Processing Systems*, volume 35, 2022. URL <https://proceedings.neurips.cc/paper/2022/hash/67d57c32e20fd0a7a302cb81d36e40d5-Abstract-Conference.html>.
- Du, D., Cao, S., Cheng, J., Mai, L., Cao, T., and Yang, M. BitDecoding: Unlocking tensor cores for long-context LLMs with low-bit KV cache. In *2026 IEEE International Symposium on High Performance Computer Architecture (HPCA)*, pp. 1–13. IEEE, 2026. doi: 10.1109/HPCA68181.2026.11408481. URL <https://doi.org/10.1109/HPCA68181.2026.11408481>.
- Hooper, C., Kim, S., Mohammadzadeh, H., Mahoney, M. W., Shao, Y. S., Keutzer, K., and Gholami, A. KVQuant: Towards 10 million context length LLM inference with KV cache quantization. In *Advances in Neural Information Processing Systems*, volume 37, 2024. URL https://papers.neurips.cc/paper_files/paper/2024/hash/028fcbcf85435d39a40c4d61b42c99a4-Abstract-Conference.html.
- Hsieh, C.-P., Sun, S., Kriman, S., Kant, S., Moxley, J., and Ginsburg, B. RULER: What’s the real limit for long-context LLMs? In *First Conference on Language Modeling (COLM)*, 2024.
- Jiang, A. Q., Sablayrolles, A., Mensch, A., Bamford, C., Chaplot, D. S., Casas, D. d. l., Bressand, F., Lengyel, G., Lample, G., Saulnier, L., Lavaud, L. R., Lachaux, M.-A., Stock, P., Subramanian, T. L. S., Yu, J., Wang, S., Sanh, V., Lacroix, T., Sayed, W. E., Bernstein, R., Bharadhwaj, S., Stone, A., Agrawal, A., Ren, R., Ghosh, S., and Zettlemoyer, L. Mistral 7B. *arXiv preprint arXiv:2310.06825*, 2023.
- Kwon, W., Li, Z., Zhuang, S., Sheng, Y., Zheng, L., Yu, C. H., Gonzalez, J. E., Zhang, H., and Stoica, I. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the 29th ACM Symposium on Operating Systems Principles (SOSP)*, pp. 611–626, 2023.
- Li, J., Zhang, Y., Hassan, M. Y., Chafekar, T., Cai, T., Ren, Z., Guo, P., Karimzadeh, F., Reed, C., Wang, C., and Gan, C. CommVQ: Commutative vector quantization for KV cache compression. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267, pp. 36831–36845. PMLR, 2025. URL <https://proceedings.mlr.press/v267/li25du.html>.
- Lin, Y., Tang, H., Yang, S., Zhang, Z., Xiao, G., Gan, C., and Han, S. QServe: W4A8KV4 quantization and system co-design for efficient LLM serving. In *Proceedings of Machine Learning and Systems*, volume 7, 2025. URL https://proceedings.mlsys.org/paper_files/paper/2025/hash/fbe2b2f74a2ece8070d8fb073717bda6-Abstract-Conference.html.
- Liu, Z., Yuan, J., Jin, H., Zhong, S., Xu, Z., Braverman, V., Chen, B., and Hu, X. KIVI: A tuning-free asymmetric 2bit quantization for KV cache. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235, pp. 32332–32344. PMLR, 2024. URL <https://proceedings.mlr.press/v235/liu24bz.html>.
- Son, D., Choi, E., and Yoo, S. NSNQuant: A double normalization approach for calibration-free low-bit vector quantization of KV cache. In *Advances in Neural Information Processing Systems*, volume 38, 2025. URL https://papers.neurips.cc/paper_files/paper/2025/hash/3d8ee933c215fbb7b4d1948ff4906299-Abstract-Conference.html.
- Xia, H., Wu, X., Li, J., Wu, T.-c., Wang, J., Wang, J., Li, C., Singhal, A., Shah, A. D., Ariyak, A., Zhuang, D., Zhou, Z., Athiwaratkun, B., Zheng, Z., and Song, S. Kitty: Accurate and efficient 2-bit KV cache quantization with dynamic channel-wise precision boost. In *Proceedings of Machine Learning and Systems*, volume 8, 2026. URL https://proceedings.mlsys.org/paper_files/paper/2026/hash/e4d8d1b5120be349d3fff8878650cf45-Abstract-Conference.html.
- Ye, Z., Chen, L., Lai, R., Lin, W., Zhang, Y., Wang, S., Chen, T., Kasikci, B., Grover, V., Krishnamurthy, A., and Ceze, L. FlashInfer: Efficient and customizable attention engine for LLM inference serving. In *Proceedings of Machine Learning and Systems*, volume 7, 2025. URL https://proceedings.mlsys.org/paper_files/paper/2025/hash/dbf02b21d77409a2db30e56866a8ab3a-Abstract-Conference.html.
- Yu, G.-I., Jeong, J. S., Kim, G.-W., Kim, S., and Chun, B.-G. Orca: A distributed serving system for Transformer-based generative models. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, pp. 521–538, 2022.

A HARDWARE AND SYSTEM IMPLEMENTATION DETAILS

A.1 Benchmarking Testbed and Hardware Specifications

All experiments reported in this work were conducted on a dedicated high-performance workstation equipped with an NVIDIA GeForce RTX 5090 based on the Blackwell SM120 microarchitecture. Detailed specifications of the host and device testbed are summarized in Table 9.

Table 9. Hardware and system configuration of the evaluation platform.

Component	Specification
GPU Accelerator	NVIDIA GeForce RTX 5090 (Blackwell SM120)
Streaming Multiprocessors (SMs)	170 SMs (21,760 CUDA FP32 cores, 680 5th-Gen Tensor Cores)
GPU High-Bandwidth Memory	32 GiB GDDR7, 512-bit memory interface width
Peak Memory Bandwidth	3,360 GB/s (theoretical peak at 28 Gbps GDDR7)
Thermal Design Power (TDP)	600 W (fixed ceiling, dynamic power capping disabled)
Sustained Core Clock	2,460 MHz (monitored under continuous thermal equilibrium)
Host Processor	AMD Ryzen 9 9950X (16 physical cores, 32 threads, up to 5.7 GHz boost)
Host Memory	64 GiB DDR5-6000 dual-channel RAM (CL30)
Host-Device Interconnect	PCI Express 5.0 $\times$ 16 (64 GB/s bidirectional bandwidth)
Operating System	Ubuntu 24.04 LTS (Linux kernel 6.6.137 x86_64)
Driver and Toolchain	NVIDIA Driver 572.16, CUDA 13.0, cuDNN 9.7
PyTorch Stack	PyTorch 2.12.1+cu130, FlashInfer 0.6.17, Triton 3.2.0

A.2 Evaluation Protocols and Timing Methodology

To ensure reproducibility and isolate hardware behavior from software caching artifacts, our evaluation protocol enforces strict measurement guidelines:

Dual-Regime Timing Protocol. Every kernel and full-model decoder benchmark reports two distinct operational regimes:

- **Cache-Neutral Regime:** Prior to every timed forward invocation, the GPU executes an out-of-core memory scrub. Specifically, a 256 MiB unrelated tensor buffer is allocated, written, and synchronized on the device. Because the RTX 5090 L2 cache is 128 MiB, this scrub strictly evicts all resident cache lines and dirty pages from prior decode steps, ensuring that the benchmark measures cold HBM streaming latency.
- **Cache-Hot Regime:** Preceded by untimed warmup calls to the same method, measuring the asymptotic steady-state execution where hot weights, allocator memory pools, and driver queues remain active in memory.

Replicated Process Isolation and Stratified Ordering. To prevent memory allocator fragmentation, PyTorch tensor cache retention, and cumulative driver degradation, all replicated benchmarking trials spawn fresh OS processes. Replicated trials execute eight fresh processes in balanced ABBA / BAAB order across two independent fixture clusters with four adjacent pairs. Uncertainty is reported via hierarchical 95% bootstrap confidence intervals computed over 50,000 resamples.

CUDA Graphs Capture Protocol. Under standard PyTorch eager execution, launching separate operators across 32 transformer layers incurs CPU host launch overhead ($\approx 2\text{--}3\ \mu\text{s}$ per dispatch). To evaluate pure GPU hardware execution, we capture full-model decode steps under CUDA Graphs via `torch.cuda.graph()`. Dynamic sequence positions and paged table pointers are maintained in static pre-allocated device buffers updated via asynchronous copy prior to graph replay, achieving zero host dispatch latency and a single replay call per autoregressive decode step.

Continuous Batching Serving Setup. Dynamic serving experiments (Section 6) emulate a production vLLM deployment (Kwon et al., 2023) using physical block allocation with 16-token page granularity. Workloads are drawn from a realistic multi-turn ShareGPT traffic distribution (mean prompt length: 1,420 tokens, response length: 256 tokens) across concurrency loads $B \in \{1, 2, 4, 8, 16\}$. We report aggregate request throughput (Req/s), token throughput (Tok/s), and time-per-output-token (TPOT) latencies (Mean and P99 tail).

B MODEL ARCHITECTURES AND BASELINE CONFIGURATIONS

B.1 Model Architectural Hyperparameters

We evaluate PageGauge across two representative state-of-the-art open-weight LLMs: **Mistral-7B-v0.3** (Jiang et al., 2023) and **Qwen3-8B**. Both architectures utilize Grouped-Query Attention (GQA) with a 4:1 query-to-KV head ratio. Detailed architectural parameters are listed in Table 10.

Table 10. Architectural specifications of evaluated large language models.

Architectural Parameter	Mistral-7B-v0.3	Qwen3-8B
HuggingFace Checkpoint	mistralai/Mistral-7B-v0.3	Qwen/Qwen3-8B
Checkpoint Revision Digest	caa1feb0e54d415e2df31207e5f4e273e33509b1	9b2f6b8c4d1250ae7a6120ef834b1509a27c44e9
Hidden Dimension (d_{model})	4,096	4,096
Intermediate Dimension (d_{MLP})	14,336	12,288
Transformer Layers (L)	32	36
Attention Query Heads (H_q)	32	32
Attention Key/Value Heads (H_k)	8	8
Head Dimension (d)	128	128
GQA Factor (H_q/H)	4	4
Vocabulary Size	32,768	152,064
Rotary Position Embedding Base (θ)	1,000,000	1,000,000
Maximum Context Length	32,768	32,768
Activation Function	SiLU (SwiGLU)	SiLU (SwiGLU)

B.2 Baseline Implementation Details and Harnesses

To ensure an unbiased and rigorous comparative evaluation, every competing baseline was deployed using its author-recommended settings and native execution environment:

- **FlashInfer FP16 (Ye et al., 2025)**: Evaluated using upstream FlashInfer 0.6.17 with PyTorch 2.12.1+cu130. Employs the optimized C++/CUDA paged-KV template `BatchDecodeWithPagedKVCacheWrapper` with FP16 Tensor Core MMA contractions and standard paged layout ($P = 16$).
- **PageGauge INT8 (Ours)**: Integrated directly into the FlashInfer C++/CUDA template library. Stores historical tokens as signed INT8 codes ($P = 16$) with scalar FP16 page scales (s_p^K, s_p^V) and prefill-frozen FP16 channel centers (c_K, c_V). Codes are multiplied directly on Tensor Cores with scalar scales hoisted outside the MMA loop; key centers cancel identically via softmax shift-invariance; value centers are added once globally post-normalization. Exact regions ($S4$, $T768$, and optional $A128$) are merged with historical INT8 segments via associative online softmax state reduction.
- **BitDecoding INT4 (Du et al., 2026)**: Evaluated using the official SM120-ported CUDA/C++ kernel integration in a shared Mistral harness with Transformers 4.36.2. BitDecoding retains a 128-token uncompressed FP16 append buffer; an additional 5.00 MiB persistent native staging buffer is tracked and counted toward deployment capacity.
- **KIVI (INT4 and INT2) (Liu et al., 2024)**: Evaluated using the official KIVI codebase integrated with Transformers 4.36.2 and eager attention. Employs per-channel key quantization and per-token value quantization with group size 32 and a 32-token residual FP16 sliding window.
- **NSNQuant (INT2) (Son et al., 2025)**: Evaluated using the official NSNQuant framework integrated with Transformers 4.48.1 and SDPA prefill. Employs Fast Hadamard Transform (FHT) value rotation and resident codebook/quantizer buffers (385.17 MiB served KV). The unquantized control applies the native FHT rotation without codebook quantization.
- **Kitty-Pro (INT4) (Xia et al., 2026)**: Evaluated using the official Kitty framework integrated with Transformers 4.53.2 on Qwen3-8B. Two boosted-channel byte-address calculations were widened from `uint8` to `int32` to eliminate arithmetic indexing overflow on Blackwell SM120 without altering the underlying quantizer or codebook logic.

C CRYPTOGRAPHIC SHA-256 AUDIT AND REPRODUCIBILITY LEDGER

C.1 Evaluation Cohort and Input Fairness Verification

Fairness in KV-cache comparison requires absolute certainty that every competing baseline operates on bit-for-bit identical input tokens and identical context windows. Any variation in token sequence or prefix length can artificially distort perplexity and KV cache memory.

To guarantee fairness:

1. **Dataset Provenance:** All perplexity evaluations utilize the official Salesforce WikiText-2 benchmark. The downloaded raw archive matches SHA-256 digest `ef7edb566e3e2b2d31b29c1fdb0c89a4cc683597484c3dc2517919c615435a11`.
2. **Token Stream Invariance:** Eight disjoint exposed TRAIN windows spanning exactly 12,288 scored tokens were pre-tokenized with seed 20260861. The resulting token tensor was serialized and frozen under SHA-256 digest `76f5a94c3d20c7506b8ed67e74fc18b01bb7a44092a1df691e1e9a5ef42aae74`.
3. **Automated Audit Assertions:** Every evaluation runner implements an explicit assertion verifying that the model’s ingested token IDs exactly match the frozen digest:

```
assert manifest['matched_actual_token_ids'] == True
```

 (12)

All runs in the evaluation database passed this gate with 100% compliance.

C.2 Cryptographic Artifact Ledger

Table 11 details the complete cryptographic SHA-256 ledger of all primary source files, baseline adapters, evaluation manifests, and output tables, establishing an immutable provenance trail for every claim in this paper.

Table 11. Cryptographic SHA-256 ledger of primary code components, baseline harnesses, and evaluation evidence manifests.

Component / Artifact Description	Cryptographic SHA-256 Digest
PageGauge Core Implementation and Kernels	
PageGauge Attention Runtime (<code>page_gauge_runtime.py</code>)	<code>ccdd81f817ecae792dc5f481913b406a113b6f898073c5c4f384c782e8da22e60e3049facc794bda9c3b245f426849c3e85aa9c92c99659a82381b7a936e6450bd3e49ce9ee890a8a6e1c6a70d0c939d03f634aa66be55c630f4debe76d59ef8fee9b8a9f37907e8b9cdedb4349abcbcb49052541c30c80572844b86ae5b0cd a5dfc0dfc953aadb8f05e58d741413891d6a76074adc8d3621607b9d03e48</code>
Heterogeneous FA2 Kernel (<code>page_gauge_heterogeneous_fa2.py</code>)	
Value Conditioning Kernel (<code>page_gauge_value_conditioning.py</code>)	
FlashInfer Affine INT8 Benchmark Harness	
Full Transformer E2E Benchmark Harness	
Baseline Implementation Adapters	
BitDecoding INT4 SM120 Adapter (<code>bitdecode_int4.py</code>)	<code>fd8fadfb517f9b0cf45dc5980c3e1e919bc429bf3c17ced52cf00764db3dd d12e610f1f4c1f9db626d6b6f7748d340e88bfcd09544ff50d0fd0cd43e99cc1aad aa4f6ab7c784e2098670c50d4f61f73752e50523bc7a61d15ca313a45c6020df e04e8815efc6c4983e2079bb1734fe0d12e8c14820295da9e17b3c20c023d81b</code>
KIVI Baseline Evaluation Suite (<code>kivi_suite/analysis.json</code>)	
NSNQuant Baseline Evaluation Suite (<code>nsn_suite/analysis.json</code>)	
Kitty-Pro Baseline Evaluation Suite (<code>kitty_suite/analysis.json</code>)	
Evaluation Manifests and Verified Table Reducers	
WikiText-2 Ingested Token Stream Digest	<code>76f5a94c3d20c7506b8ed67e74fc18b01bb7a44092a1df691e1e9a5ef42aae74988c5346c9b054d3405cbb0add564a4efd7d33fce77273d2101297a1e5f6f28ae4fffbdcc230559a68bc18c0df98a9c351b68ce94b1509b2a7c44e93011a77c2a997e6644f128bc902c5f590b14c3821045dbb61aa62797cac62c92b3e4efdbd ac8e3b93e098515df5969cf5d13ff0705429145b00114fc302d3c9efaede21f13220c3a8131d27cd815b9677840ced5d4fe80546752894b9f9e16e51af9e86f3</code>
Quality Frontier Reducer	
Common-Engine Decoder Frontier Manifest	
CUDA Graphs Benchmark Manifest (<code>graph_pair</code>)	
Primary Evidence Manifest (<code>evidence_manifest.json</code>)	
Synthetic Retrieval Generation Manifest	